

Core Java Interview Questions

Comprehensive Q&A Guide for Java Developers

1. Why is Java platform independent?

Answer: Java is platform independent because Java code is compiled into bytecode (.class file), which runs on the Java Virtual Machine (JVM). Since every operating system has its own JVM, the same bytecode can run anywhere.

"Write Once, Run Anywhere (WORA)."

2. What is JVM, JRE, and JDK?

JDK	JRE	JVM
Java Development Kit	Java Runtime Environment	Java Virtual Machine
Used to develop Java programs	Used to run Java programs	Executes bytecode
Contains JRE + development tools	Contains JVM + libraries	Part of JRE

3. Difference between == and equals()

==	equals()
Compares references	Compares object content
Used for primitives and references	Used for objects
Can be overridden	Defined in Object class

Example:

```
String s1 = new String("Java");
String s2 = new String("Java");

System.out.println(s1 == s2);      // false
System.out.println(s1.equals(s2)); // true
```

4. What is the difference between String, StringBuilder, and StringBuffer?

String	StringBuilder	StringBuffer
Immutable	Mutable	Mutable
Thread-safe (immutable)	No	Yes
Slower for modifications	Fast	Slower than StringBuilder

5. Why is String immutable?

Answer: String is immutable for:

- Security
- Thread safety
- String pool optimization
- Better performance

6. What is method overloading?

Answer: Having multiple methods with the same name but different parameters.

Example:

```
add(int a, int b)
add(double a, double b)
```

Compile-time polymorphism.

7. What is method overriding?

Answer: A child class provides its own implementation of a method already defined in the parent class.

Run-time polymorphism.

8. Difference between overloading and overriding

Overloading	Overriding
Same class	Parent & child class
Different parameters	Same parameters
Compile time	Runtime

9. What is encapsulation?

Answer: Wrapping data and methods into one class and protecting data using private variables with getters and setters.

10. What is inheritance?

Answer: Inheritance allows one class to acquire properties and methods of another class using extends.

11. Why doesn't Java support multiple inheritance?

Answer: To avoid the Diamond Problem, where a class inherits the same method from multiple parent classes, causing ambiguity.

Java supports multiple inheritance through interfaces.

12. What is abstraction?

Answer: Hiding implementation details and showing only essential functionality.

Achieved using:

- Abstract class
- Interface

13. Difference between abstract class and interface

Abstract Class	Interface
Can have constructors	Cannot have constructors
Can have instance variables	Only constants
extends	implements
Supports partial abstraction	Supports complete abstraction (plus default/static methods)

14. What is a constructor?

Answer: A special method used to initialize objects.

Properties:

- Same name as class
- No return type
- Called automatically when an object is created

15. Can we override constructors?

Answer: No. Constructors are not inherited, so they cannot be overridden.

16. Can static methods be overridden?

Answer: No. Static methods belong to the class, not to objects. They can only be hidden, not overridden.

17. What is the final keyword?

- **final variable** → Cannot be changed.
- **final method** → Cannot be overridden.
- **final class** → Cannot be inherited.

18. Difference between Array and ArrayList

Array	ArrayList
Fixed size	Dynamic size
Stores primitives & objects	Stores objects (primitives via wrappers)
Faster	More flexible

19. Difference between HashMap and HashSet

HashMap	HashSet
Key-value pairs	Only unique values
Keys unique	Values unique
Allows one null key	Allows one null element

20. Difference between HashMap and TreeMap

HashMap	TreeMap
No order	Sorted by keys
$O(1)$ average	$O(\log n)$
Uses hash table	Uses Red-Black Tree

21. Difference between HashSet and TreeSet

HashSet	TreeSet
No order	Sorted
$O(1)$ average	$O(\log n)$
Hash table	Red-Black Tree

22. What is an exception?

Answer: An exception is an event that interrupts the normal execution of a program.

Examples: `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`

23. Difference between checked and unchecked exceptions

Checked	Unchecked
Checked at compile time	Checked at runtime
Must be handled	Handling is optional
Example: <code>IOException</code>	Example: <code>NullPointerException</code>

24. What is the difference between throw and throws?

throw	throws
Used to explicitly throw an exception	Declares exceptions a method may throw
Used inside a method	Used in the method signature

25. What is the difference between List, Set, and Map?

List	Set	Map
Allows duplicates	No duplicates	Key-value pairs
Maintains insertion order	Usually no duplicates; ordering depends on implementation	Keys are unique
Example: <code>ArrayList</code>	Example: <code>HashSet</code>	Example: <code>HashMap</code>

Tip for Interviews

For each answer, follow this structure:

1. **Definition** (1 sentence)
2. **Key features** (2–3 points)
3. **Example** (if applicable)

This approach makes your answers clear, concise, and easy for interviewers to follow.

Deeper Core Java Questions (2–5 Years Experience)

If you're preparing for 2–5 years Java Developer interviews (Cognizant, TCS, Accenture, Capgemini, Infosys, Wipro, etc.), interviewers often ask deeper Core Java questions like these.

26. Why is String immutable? (Detailed)

Answer: String is immutable because once an object is created, its value cannot be changed.

Why?

- Security (URL, username, passwords cannot be modified)
- Thread-safe
- String Constant Pool optimization
- Better hashing performance

"String is immutable to improve security, thread safety, performance, and memory optimization."

27. Why are wrapper classes immutable?

Answer: Wrapper classes like Integer, Double, and Boolean are immutable because:

- They are thread-safe.
- They can be cached (`Integer.valueOf()`).
- They can safely be used as keys in maps.

28. How does HashMap work internally?

Answer: When we execute: `map.put("Java", 100);`

HashMap performs these steps:

1. Calls `hashCode()` on the key.
2. Calculates the bucket index.
3. Stores the entry in that bucket.
4. If another key maps to the same bucket (collision), it stores multiple entries in that bucket.
5. Java 8 converts long collision chains into a balanced tree when appropriate for better performance.

Time Complexity:

- Search → $O(1)$ average
- Insert → $O(1)$ average
- Worst case → $O(n)$, or $O(\log n)$ when treeified

29. What is `hashCode()` and `equals()`?

- **`equals()`**: Checks whether two objects are logically equal.
- **`hashCode()`**: Returns an integer hash value used for efficient storage and lookup in hash-based collections.

Rule: If two objects are equal according to `equals()`, they must have the same `hashCode()`.

30. Why should `equals()` and `hashCode()` be overridden together?

Because collections like `HashMap` and `HashSet` first use `hashCode()` to find a bucket and then use `equals()` to compare objects within that bucket.

If only one is overridden, duplicate detection and lookups may fail.

31. What is the String Constant Pool?

```
String s1 = "Java";  
String s2 = "Java";
```

Both variables point to the same object in the String Pool.

```
String s3 = new String("Java");
```

This creates a new object in heap memory.

32. What is the difference between heap and stack memory?

Stack	Heap
Stores local variables	Stores objects
Faster	Slower
Automatically cleaned after method execution	Managed by Garbage Collector
One stack per thread	Shared by all threads

33. What is Garbage Collection?

Answer: Garbage Collection automatically removes objects that are no longer referenced, freeing heap memory.

Advantages:

- Prevents memory leaks.
- Automatically manages memory.

34. Can we force Garbage Collection?

```
System.gc();
```

No. It only requests the JVM to perform garbage collection. The JVM decides whether and when to run it.

35. What is the difference between final, finally, and finalize()?

final	finally	finalize()
Keyword	Block	Method
Prevents modification	Executes after try/catch	Called by GC before object removal (deprecated)

36. Why is Java pass-by-value?

Java always passes a copy of the variable's value.

- For primitives, the value itself is copied.
- For objects, the reference value is copied (not the object).

So Java is always pass-by-value.

37. Difference between Comparable and Comparator

Comparable	Comparator
Natural sorting	Custom sorting
compareTo()	compare()
Defined inside the class	Separate class or lambda

38. Why is multiple inheritance not supported?

To avoid the Diamond Problem.

If two parent classes have the same method, the compiler cannot determine which one to inherit. Java supports multiple inheritance through interfaces.

39. What is marker interface?

An interface with no methods or fields.

Examples: Serializable, Cloneable

Used to provide metadata to the JVM or frameworks.

40. What is serialization?

Serialization converts an object into a byte stream so it can be:

- Saved to a file.
- Sent over a network.

Deserialization converts the byte stream back into an object.

41. What is the difference between fail-fast and fail-safe iterators?

Fail-Fast	Fail-Safe
Throws ConcurrentModificationException	Does not throw the exception
Uses original collection	Uses a copy of the collection
Example: ArrayList iterator	Example: CopyOnWriteArrayList

42. Why is ArrayList faster than LinkedList?

ArrayList: Stores elements in contiguous memory. Fast random access using indexes (`get(index)`).

LinkedList: Stores elements as nodes connected by links. Slower random access because it traverses from the beginning or end.

43. What happens when we create an object?

```
Student s = new Student();
```

Steps:

1. JVM loads the class (if not already loaded).
2. Memory is allocated in the heap.
3. Instance variables are initialized.
4. Constructor is executed.
5. Reference is stored in the stack.

44. What is reflection?

Reflection allows Java to inspect and manipulate classes, methods, constructors, and fields at runtime.

It is commonly used by frameworks like Spring and Hibernate.

45. What is the difference between instanceof and getClass()?

- `obj instanceof Animal`: Checks whether the object belongs to the specified class or one of its subclasses.
- `obj.getClass()`: Returns the object's exact runtime class.

46. What is the transient keyword?

`transient` prevents a field from being serialized.

```
transient String password;
```

Useful for sensitive data like passwords.

47. What is the volatile keyword?

`volatile` ensures that changes to a variable made by one thread are immediately visible to other threads.

It improves visibility but does not make compound operations (like incrementing) atomic.

48. Why can't we instantiate an abstract class?

An abstract class may contain abstract methods without implementations, making it an incomplete class. Therefore, Java does not allow creating its objects directly. A subclass must provide implementations for the abstract methods.

49. Can we have a constructor in an abstract class?

Yes. Constructors in abstract classes are used to initialize common fields when a subclass object is created.

50. Can we override a private method?

No. Private methods are not inherited by subclasses, so they cannot be overridden.

Additional Concepts

51. What is autoboxing and unboxing?

Autoboxing: The automatic conversion that the Java compiler makes between the primitive types and their corresponding object wrapper classes. For example, converting an `int` to an `Integer`, a `double` to a `Double`, etc.

Unboxing: The reverse process of autoboxing. It is the automatic conversion of wrapper class objects back to their corresponding primitive types.

Example:

```
List<Integer> list = new ArrayList<>();  
list.add(10); // Autoboxing: int 10 is converted to Integer object  
  
int num = list.get(0); // Unboxing: Integer object is converted back to int
```

52. What are garbage value collectors (Garbage Collection algorithms)?

While Garbage Collection (GC) is the process of reclaiming memory, a Garbage Collector is the specific JVM component/algorithm that executes this process. They track live objects and clear "garbage" (unreachable) values from memory.

Common JVM Garbage Collectors:

- **Serial GC:** Designed for single-threaded environments and small datasets.
- **Parallel GC:** Uses multiple threads to speed up garbage collection. It is the default in older Java versions (like Java 8).

- **G1 GC (Garbage First):** The default GC in newer Java versions (Java 9+). It partitions the heap into regions and prioritizes collecting regions with the most garbage.
- **ZGC / Shenandoah:** Modern, low-latency garbage collectors designed for very large heaps with minimal pause times.

★ Frequently Asked Advanced Interview Topics to Prepare:

- Explain the Java Memory Model (JMM).
- What are strong, weak, soft, and phantom references?
- How does the JVM load classes?
- What is the class loader hierarchy?
- What is the difference between shallow copy and deep copy?
- Explain object cloning.
- Why is Object the parent of all Java classes?
- What are immutable classes, and how do you create one?
- What are functional interfaces, and why are they useful?
- Explain the difference between Collection and Collections.
- Explain the difference between Comparable, Comparator, and equals().

Mastering these questions will prepare you for many intermediate Java Core interviews and also provide a strong foundation for Spring Boot discussions.